



CMG METALHIDRÁULICA S.L.

CMG Telematics

Manual Técnico de Desarrollo e Instalación

Versión: 1.0.0 — Piloto

Fecha: Marzo 2026

VPS: 213.210.20.183

Stack: FastAPI + PostgreSQL/TimescaleDB + Next.js 16 + PWA

Hardware: Teltonika FMC650 → IFM CR2530 CAN J1939

CONFIDENCIAL — USO INTERNO

Índice de Contenidos

Capítulo 1 — Arquitectura del Sistema	3
Stack tecnológico	3
Infraestructura VPS	3
Puertos y servicios	4
Flujo de datos	4
Capítulo 2 — Instalación del Servidor VPS	6
Requisitos del sistema	6
PostgreSQL 16 + TimescaleDB	6
Redis	7
Backend Python	7
Frontend Node.js	8
Caddy Reverse Proxy	8
Servicios systemd	9
Capítulo 3 — Backend FastAPI	11
Estructura del proyecto	11
Modelos de datos	11
Referencia de endpoints API	13
Autenticación JWT	15
Arquitectura multi-tenant	15
Sistema de alertas	16
WebSocket tiempo real	16
Capítulo 4 — Frontend Next.js	17
Estructura del proyecto	17
Páginas implementadas	17
Componentes principales	18
Proxy API y WebSocket	19
Configuración PWA	19
Ciclo de build y deploy	20
Capítulo 5 — Protocolo Teltonika FMC650	21
Protocolo Codec 8	21
Servidor TCP asyncio	22
Tabla de IDs de IO	23
Comandos remotos DOUT	24
Variable Maps	24
Capítulo 6 — Configuración Hardware FMC650	25
Configuración del dispositivo	25
Bus CAN e IFM CR2530	26

Capítulo 7 — Administración de la Plataforma	27
Jerarquía de tenants	27
Roles y permisos	27
Configuración paso a paso	28
Capítulo 8 — Operación y Mantenimiento	30
Comandos de operación	30
Logs y monitorización	30
Acceso a base de datos	31
Resolución de problemas	31

Arquitectura del Sistema

Stack Tecnológico

Backend

FastAPI	0.115.6
Uvicorn + uvloop	0.32.1
SQLAlchemy asyncio	2.0.36
asynctpg	0.30.0
Alembic	1.14.0
python-jose JWT	3.3.0
passlib bcrypt	1.7.4
redis asyncio	5.2.1
aiomqtt	2.3.0
Python	3.11+

Frontend

Next.js	16.2.0
React	19.2.4
TypeScript	5.x
TailwindCSS	4.x
Recharts	3.8.0
Leaflet	1.9.4
next-pwa	5.6.0
lucide-react	0.577.0
Node.js	20+

Base de Datos

PostgreSQL	16 (nativo)
TimescaleDB	extensión activa

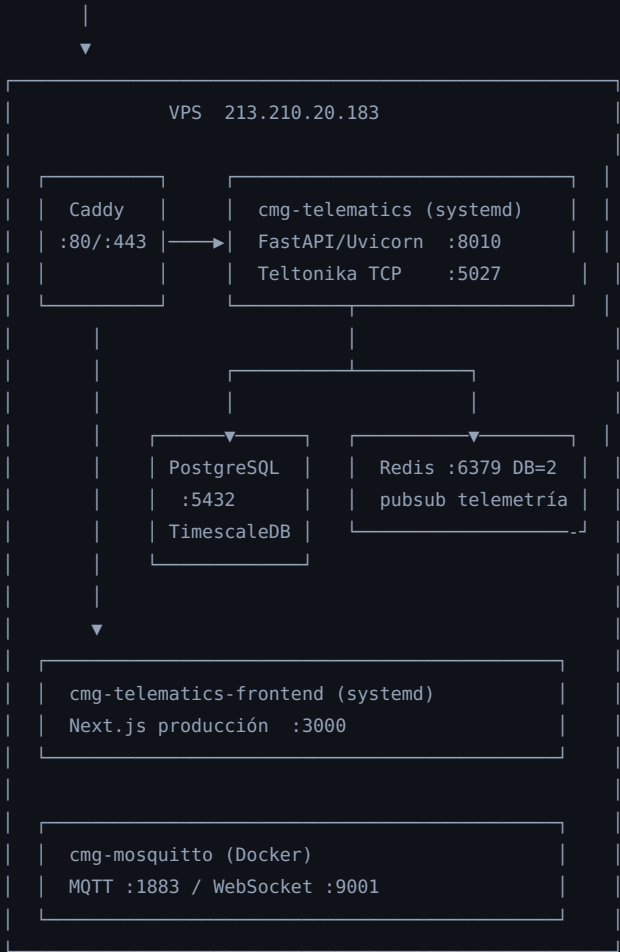
Redis	nativo, DB=2
Hypertable	telemetry_record
Retención datos	2 años
Compresión	chunks > 7 días

Infraestructura

VPS IP	213.210.20.183
OS	Ubuntu 22.04 LTS
Reverse proxy	Caddy
MQTT broker	Mosquitto (Docker)
Hardware	Teltonika FMC650
CAN sensor	IFM CR2530

Diagrama de Infraestructura

Internet / FMC650 en campo



FMC650 —TCP:5027→ Teltonika Server → PostgreSQL
→ Redis pubsub → WebSocket /ws/fleet

```
Cliente —HTTPS:443—> Caddy —> Next.js :3000
      —> FastAPI :8010 (/api/*, /ws/*, /health)
```

Puertos y Servicios

PUERTO	PROTOCOLO	SERVICIO	ACCESO	DESCRIPCIÓN
80	HTTP	Caddy	Público	Redirect o proxy HTTP
443	HTTPS	Caddy	Público	Reverse proxy TLS (Let's Encrypt)
3000	HTTP	Next.js	Localhost	Frontend Next.js producción
5027	TCP	Teltonika Server	Público	Protocolo Codec 8, FMC650 en campo
5432	TCP	PostgreSQL 16	Localhost	Base de datos principal + TimescaleDB
6379	TCP	Redis	Localhost	Pubsub telemetría, caché (DB=2)
8010	HTTP	FastAPI	Localhost	API REST + WebSocket
1883	TCP	Mosquitto (Docker)	Localhost	MQTT broker
9001	WS	Mosquitto (Docker)	Localhost	MQTT sobre WebSocket

Seguridad

Los puertos 5432 (PostgreSQL) y 6379 (Redis) NUNCA deben estar expuestos al exterior. Solo accesibles desde localhost. Verificar con: `ss -tlnp | grep -E "5432|6379"`

Flujo de Datos

Telemetría en tiempo real

1. El dispositivo FMC650 mantiene una conexión TCP persistente al puerto 5027
2. Envía paquetes Codec 8 cada N segundos (configurable)
3. El servidor TCP valida el IMEI contra la base de datos
4. Parsea el AVL record: GPS, IO digitales, IO analógicas, CAN J1939
5. Persiste en `telemetry_record` (hypertable TimescaleDB)
6. Publica en Redis pubsub canal `telemetry:{device_id}`
7. El WebSocket `/ws/fleet` recibe del pubsub y retransmite a clientes web
8. El evaluador de alertas (`_check_alerts`) comprueba reglas en background
9. El evaluador de geocercas (`_check_geofences`) detecta entradas/salidas
10. El servidor responde con ACK (número de registros guardados)

Comandos remotos DOUT

1. El usuario pulsa el toggle DOUT en la UI web (con confirmación modal)
2. POST `/api/v1/commands/send` con imei, output (DOUT1-4), value
3. FastAPI encola el comando en `_command_queues[imei]`
4. Tras el próximo ACK al dispositivo, el servidor vacía la cola y envía el comando ASCII
5. El FMC650 ejecuta `setdigout X Y T` y activa/desactiva el relay
6. El estado se confirma en el siguiente paquete de telemetría

Instalación del Servidor VPS

Requisitos del Sistema

Sistema operativo	Ubuntu 22.04 LTS (recomendado)
CPU	2+ vCPUs
RAM	4 GB mínimo, 8 GB recomendado
Disco	40 GB SSD (TimescaleDB crece con el tiempo)
Red	IP pública estática, puertos 80, 443, 5027 abiertos
Python	3.11 o superior
Node.js	20 LTS o superior

PostgreSQL 16 + TimescaleDB

```
# Añadir repositorio PostgreSQL
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc | gpg --dearmor -o /usr/share/keyrings/postgre
echo "deb [signed-by=/usr/share/keyrings/postgresql.gpg] https://apt.postgresql.org/pub/repos/apt $(lsb_rel

# Añadir repositorio TimescaleDB
curl -fsSL https://packagecloud.io/timescale/timescaledb/gpgkey | gpg --dearmor -o /usr/share/keyrings/time
echo "deb [signed-by=/usr/share/keyrings/timescaledb.gpg] https://packagecloud.io/timescale/timescaledb/ubun

apt-get update
apt-get install -y postgresql-16 timescaledb-2-postgresql-16

# Configurar PostgreSQL
timescaledb-tune --pg-config=/usr/lib/postgresql/16/bin/pg_config --yes
systemctl restart postgresql

# Crear usuario y base de datos
sudo -u postgres psql << EOF
CREATE USER cmg WITH PASSWORD 'cmg_pilot_2024';
CREATE DATABASE cmg_telematics OWNER cmg;
GRANT ALL PRIVILEGES ON DATABASE cmg_telematics TO cmg;
EOF

# Habilitar TimescaleDB en la base de datos
PGPASSWORD=cmg_pilot_2024 psql -U cmg -d cmg_telematics -c "CREATE EXTENSION IF NOT EXISTS timescaledb;"
```

Configuración postgresql.conf (ajustes clave)

```
# /etc/postgresql/16/main/postgresql.conf
shared_preload_libraries = 'timescaledb'
max_connections = 100
shared_buffers = 1GB           # 25% de RAM
effective_cache_size = 3GB     # 75% de RAM
work_mem = 64MB
maintenance_work_mem = 256MB
wal_level = replica
```

```
max_wal_size = 2GB
```

Redis

```
apt-get install -y redis-server

# Configurar Redis
cat >> /etc/redis/redis.conf << EOF
bind 127.0.0.1
maxmemory 512mb
maxmemory-policy allkeys-lru
EOF

systemctl enable redis-server
systemctl start redis-server

# Verificar
redis-cli ping # → PONG
```

Backend Python

```
# Clonar repositorio
git clone https://github.com/camaro734/cmg-telematics.git /opt/cmg-telematics

# Crear entorno virtual
cd /opt/cmg-telematics/backend
python3 -m venv venv
source venv/bin/activate

# Instalar dependencias
pip install -r requirements.txt

# Crear archivo de entorno
cat > /opt/cmg-telematics/.env << EOF
DATABASE_URL=postgresql+asyncpg://cmg:cmg_pilot_2024@localhost:5432/cmg_telematics
REDIS_URL=redis://localhost:6379/2
SECRET_KEY=$(python3 -c "import secrets; print(secrets.token_hex(32))")
ENVIRONMENT=pilot
TCP_HOST=0.0.0.0
TCP_PORT=5027
ACCESS_TOKEN_EXPIRE_MINUTES=60
CORS_ORIGINS=["*"]
EOF

# Aplicar migraciones (crea las 12 tablas + hypertable)
cd /opt/cmg-telematics/backend
source venv/bin/activate
alembic upgrade head

# Crear datos iniciales (admin, tenant CMG)
python scripts/seed_pilot.py

# Verificar
curl http://localhost:8010/health
```

Seed pilot

El script `seed_pilot.py` crea: tenant CMG (tipo "cmg"), usuario `admin@cmg.es / admin123` (role superadmin), y un vehículo de prueba con IMEI `352000000000001`.

Frontend Node.js

```
# Instalar Node.js 20 LTS
```



```

curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt-get install -y nodejs

# Instalar dependencias del frontend
cd /opt/cm-g-telematics/frontend
npm install

# Build de producción
npm run build
# → Genera .next/standalone y archivos estáticos

# Verificar que arranca
node node_modules/.bin/next start -p 3000 &
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/ # → 200
kill %1

```

Caddy Reverse Proxy

```

# Instalar Caddy
apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | gpg --dearmor -o /usr/share/keyrings/ca
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | tee /etc/apt/sources.list.d/caddy
apt update && apt install caddy

```

Configuración Caddyfile (`/etc/caddy/Caddyfile`)

```

# CMG Telematics – acceso por IP (piloto sin dominio)
http://213.210.20.183 {
    # Archivos estáticos (logos fabricantes)
    handle /static/* {
        reverse_proxy localhost:8010
    }
    # API REST → backend FastAPI
    handle /api/* {
        reverse_proxy localhost:8010 {
            header_up X-Real-IP {remote_host}
        }
    }
    # WebSocket tiempo real → backend
    handle /ws/* {
        reverse_proxy localhost:8010 {
            transport http { versions 1.1 }
        }
    }
    # Health, docs
    handle /health { reverse_proxy localhost:8010 }
    handle /docs* { reverse_proxy localhost:8010 }
    # Todo lo demás → frontend Next.js
    handle { reverse_proxy localhost:3000 }
}

# Con dominio propio (descomentar cuando se configure DNS):
# tudominio.com {
#     handle /api/* {
#         reverse_proxy localhost:8010 {
#             header_up X-Real-IP {remote_host}
#             header_up X-Forwarded-Proto https
#         }
#     }
#     handle /ws/* { reverse_proxy localhost:8010 { transport http { versions 1.1 } } }
#     handle { reverse_proxy localhost:3000 }
#     header {
#         Strict-Transport-Security "max-age=31536000; includeSubDomains"
#         X-Frame-Options "SAMEORIGIN"
#         X-Content-Type-Options "nosniff"
#     }
# }

```

```
systemctl enable caddy
systemctl start caddy
systemctl status caddy
```

Servicios systemd

Backend: `/etc/systemd/system/cm-g-telematics.service`

```
[Unit]
Description=CMG Telematics Backend (FastAPI + Teltonika TCP Server)
After=network.target postgresql.service redis-server.service
Wants=postgresql.service redis-server.service

[Service]
Type=simple
User=root
WorkingDirectory=/opt/cm-g-telematics/backend
EnvironmentFile=/opt/cm-g-telematics/.env
ExecStart=/opt/cm-g-telematics/backend/venv/bin/uvicorn \
    app.main:app \
    --host 0.0.0.0 \
    --port 8010 \
    --workers 1 \
    --loop uvloop
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal
SyslogIdentifier=cm-g-telematics
TimeoutStopSec=30

[Install]
WantedBy=multi-user.target
```

workers=1 es obligatorio

El servidor TCP de Teltonika usa estado en memoria (dict de conexiones activas). Con múltiples workers, cada proceso tendría su propio estado y los comandos remotos no podrían encontrar la conexión del dispositivo. Usar siempre `--workers 1`.

Frontend: `/etc/systemd/system/cm-g-telematics-frontend.service`

```
[Unit]
Description=CMG Telematics Frontend (Next.js)
After=network.target cm-g-telematics.service
Wants=cm-g-telematics.service

[Service]
Type=simple
User=root
WorkingDirectory=/opt/cm-g-telematics/frontend
Environment=NODE_ENV=production
Environment=PORT=3000
ExecStart=/usr/bin/node /opt/cm-g-telematics/frontend/node_modules/.bin/next start -p 3000
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal
SyslogIdentifier=cm-g-frontend

[Install]
WantedBy=multi-user.target
```

```
# Habilitar y arrancar ambos servicios
systemctl daemon-reload
systemctl enable cmg-telematics cmg-telematics-frontend
systemctl start cmg-telematics cmg-telematics-frontend

# Verificar estado
systemctl status cmg-telematics
systemctl status cmg-telematics-frontend

# Verificar health
curl http://localhost:8010/health
# → {"status":"ok","tcp_server":"running","db":"ok","redis":"ok","active_devices":0}
```

Backend FastAPI

Estructura del Proyecto

```

/opt/cmg-telematics/backend/
├─ venv/                                # Entorno virtual Python
├─ app/
│   ├── main.py                        # FastAPI app, lifespan, CORS, router
│   ├── core/
│   │   ├── config.py                 # Settings (pydantic-settings, lee .env)
│   │   ├── database.py               # Engine asyncio, init_db(), get_db()
│   │   └── security.py                # JWT decode, get_current_user(), require_role()
│   ├── models/
│   │   ├── tenant.py                 # Tenant (cmg/manufacturer/end_client)
│   │   ├── user.py                   # User (5 roles)
│   │   ├── vehicle.py                # Vehicle
│   │   ├── device.py                 # Device (FMC650, IMEI)
│   │   ├── telemetry.py               # TelemetryRecord (hypertable TimescaleDB)
│   │   ├── alert_rule.py              # AlertRule (umbrales configurables)
│   │   ├── alert_log.py               # AlertLog (alertas disparadas)
│   │   ├── variable_map.py            # VariableMap (IO → unidades ingeniería)
│   │   ├── maintenance.py             # MaintenanceTask + MaintenanceLog
│   │   ├── geofence.py                # Geofence + GeofenceEvent
│   │   └── command_log.py             # CommandLog (historial de comandos)
│   └─ api/
│       └─ v1/
│           ├── router.py               # Incluye todos los sub-routers
│           ├── auth.py                 # POST /auth/login, /auth/refresh, GET /auth/me
│           ├── vehicles.py              # CRUD vehículos + telemetría
│           ├── telemetry.py             # Historial y stats por dispositivo
│           ├── dashboard.py             # GET /dashboard/fleet
│           ├── commands.py              # POST /commands/send, historial
│           ├── alerts.py                # Alertas activas, historial, acknowledge
│           ├── alert_rules.py           # CRUD reglas de alerta
│           ├── variable_map.py          # CRUD variable maps
│           ├── maintenance.py           # CRUD tareas mantenimiento
│           ├── trips.py                 # Detección y listado de viajes
│           ├── geofences.py             # CRUD geocercas + eventos
│           ├── ecodriving.py            # Scores de conducción eficiente
│           └── admin.py                 # CRUD tenants, users, vehicles (admin)
│   └─ services/
│       ├── teltonika/
│       │   ├── tcp_server.py           # TeltonikaServer asyncio
│       │   ├── codec8.py                # Parser/builder Codec 8
│       │   └── device_registry.py       # Redis: dispositivos online
│       ├── geofence.py                 # is_inside_geofence()
│       └── websocket.py                 # WebSocket /ws/fleet
├─ alembic/                             # Migraciones DB
└─ scripts/seed_pilot.py                 # Datos iniciales

```

Modelos de Datos

Tenant

CAMPO	TIPO	DESCRIPCIÓN
id	UUID PK	Identificador único
name	String(200)	Nombre del tenant
type	Enum	"cmg" "manufacturer" "end_client"
parent_id	UUID FK→tenant	Tenant padre (jerarquía)
active	Boolean	Activo/desactivado
brand_name	String(200)	White-label: nombre de marca
brand_color	String(7)	White-label: color hex (#1D9E75)
logo_url	String(500)	URL del logo del fabricante
custom_domain	String(200)	Dominio propio (único)

User

CAMPO	TIPO	DESCRIPCIÓN
id	UUID PK	
tenant_id	UUID FK→tenant	Tenant al que pertenece
email	String(254) UNIQUE	Email (login)
hashed_password	String(256)	bcrypt hash
full_name	String(200)	Nombre completo
role	Enum	"superadmin" "admin" "operator" "viewer" "driver"
active	Boolean	Cuenta activa

Vehicle

CAMPO	TIPO	DESCRIPCIÓN
id	UUID PK	
tenant_id	UUID FK→tenant	Cliente final propietario
manufacturer_id	UUID FK→tenant	Fabricante (para variable maps)
name	String(200)	Nombre del vehículo
license_plate	String(20)	Matrícula
vin	String(17)	VIN (número de bastidor)
active	Boolean	Activo

Device

CAMPO	TIPO	DESCRIPCIÓN
id	UUID PK	
vehicle_id	UUID FK→vehicle nullable	Vehículo asignado
imei	String(15) UNIQUE	IMEI del FMC650

model	String(50)	Modelo (default "FMC650")
online	Boolean	Conectado actualmente
last_seen	DateTime TZ	Última vez visto
firmware_version	String(20)	Versión firmware
active	Boolean	Si false, rechaza conexiones TCP

TelemetryRecord (Hypertable TimescaleDB)

Hypertable TimescaleDB
Particionado por columna `time` en chunks de 1 día. Compresión automática de chunks mayores de 7 días. Retención automática de 2 años. Índice compuesto `(device_id, time DESC)`.

CAMPO	TIPO	DESCRIPCIÓN
id	UUID PK	
time	DateTime TZ PK	Timestamp del registro (clave de partición)
device_id	UUID FK→device	Dispositivo origen
lat / lng	Float	Coordenadas GPS
altitude	SmallInt	Altitud (metros)
speed	SmallInt	Velocidad (km/h)
angle	SmallInt	Rumbo (grados, 0=Norte)
satellites	SmallInt	Satélites GPS usados
priority	SmallInt	0=low, 1=high, 2=panic
ignition	Boolean	Estado de contacto (IO ID=1)
ext_voltage_mv	Integer	Tensión alimentación mV (IO ID=66)
battery_mv	Integer	Batería interna mV (IO ID=67)
dout1..dout4	Boolean	Estado salidas digitales (IO IDs 179-182)
din1..din4	Boolean	Estado entradas digitales (IO IDs 1-4)
io_data	JSONB	Todos los IOs raw del paquete (flexible, incl. CAN J1939)

AlertRule

CAMPO	TIPO	DESCRIPCIÓN
tenant_id	UUID FK	Tenant propietario
vehicle_id	UUID FK nullable	null = aplica a toda la flota
io_key	String(50)	"ignition", "ain1_mv", "300" (CAN), "dout1"
condition	Enum	"gt" "lt" "gte" "lte" "eq" "neq"
threshold	Float	Umbral en unidades de ingeniería
scale_factor / offset	Float	Conversión raw → engineering
level	Enum	"high" "medium" "low"
cooldown_minutes	Integer	Tiempo mínimo entre disparos (default 60)

VariableMap

CAMPO	TIPO	DESCRIPCIÓN
vehicle_id	UUID nullable	Si set: excepción para vehículo específico
tenant_id	UUID nullable	Si set: plantilla de fabricante
io_key	String(20)	"ain1_mv", "io_300", "dout1"
display_name	String(200)	Nombre legible ("Presión hidráulica")
unit	String(20)	"bar", "°C", "rpm"
scale_factor / offset	Float	raw × scale + offset = valor UI
alert_high / alert_low	Float nullable	Umbral de alarma
data_type	Enum	"gauge" "counter" "boolean" "hours"

Referencia de Endpoints API

Autenticación

MÉTODO	ENDPOINT	DESCRIPCIÓN
POST	/api/v1/auth/login	email+password → JWT access_token
POST	/api/v1/auth/refresh	Renueva el token JWT
GET	/api/v1/auth/me	Perfil del usuario autenticado

Vehículos y Telemetría

MÉTODO	ENDPOINT	DESCRIPCIÓN
GET	/api/v1/dashboard/fleet	Todos los vehículos con último estado y posición
GET	/api/v1/vehicles	Lista paginada de vehículos del tenant
POST	/api/v1/vehicles	Crear vehículo (admin)
GET	/api/v1/vehicles/{id}	Detalle de vehículo
PUT	/api/v1/vehicles/{id}	Actualizar vehículo (admin)
GET	/api/v1/vehicles/{id}/last	Último registro de telemetría
GET	/api/v1/vehicles/{id}/telemetry	Historial (params: hours, bucket_minutes)
GET	/api/v1/telemetry/{dev}/history	Serie temporal (params: start, end, bucket_minutes)
GET	/api/v1/telemetry/{dev}/stats	Estadísticas: activaciones, horas motor, distancia

Comandos, Alertas, Mantenimiento

MÉTODO	ENDPOINT	DESCRIPCIÓN
POST	/api/v1/commands/send	Enviar DOUT (imei, output DOUT1-4, value, duration_seconds)
GET	/api/v1/commands/history	Historial de comandos
GET	/api/v1/alerts/active	Alertas activas no resueltas

GET	/api/v1/alerts/active/count	Número de alertas (para badge UI)
POST	/api/v1/alerts/{id}/acknowledge	Reconocer alerta
GET	/api/v1/alert-rules	Reglas de alerta del tenant
POST	/api/v1/alert-rules	Crear regla de alerta
GET	/api/v1/maintenance/tasks	Tareas de mantenimiento
POST	/api/v1/maintenance/{id}/log	Registrar mantenimiento realizado
GET	/api/v1/maintenance/summary	Resumen: vencidas, próximas a vencer
GET	/api/v1/trips	Viajes (params: vehicle_id, start, end)
GET	/api/v1/trips/{id}/track	Puntos GPS del viaje
GET	/api/v1/geofences	Geocercas del tenant
POST	/api/v1/geofences	Crear geocerca (círculo o polígono)
GET	/api/v1/variable-maps	Variable maps (params: vehicle_id, tenant_id)
GET	/api/v1/variable-maps/resolved	Mapa resuelto (plantilla + excepciones merged)

Autenticación JWT

```
# Login
POST /api/v1/auth/login
{"email": "admin@cmg.es", "password": "admin123"}

# Respuesta
{"access_token": "eyJ...", "token_type": "bearer", "user": {...}}

# Uso en requests
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# Payload del token
{"sub": "uuid-usuario", "exp": 1740000000, "role": "superadmin", "tenant_id": "uuid"}
```

Arquitectura Multi-Tenant

La jerarquía de tenants es: **CMG (operador)** → **Fabricante** → **Cliente Final**. Todos los queries filtran por tenant_id del usuario autenticado. Nunca se devuelven datos de un tenant diferente.

```
# CORRECTO – siempre filtrar por tenant
result = await db.execute(
    select(Vehicle)
    .where(Vehicle.tenant_id == current_user.tenant_id)
    .where(Vehicle.active == True)
)

# INCORRECTO – NUNCA hacer esto
result = await db.execute(select(Vehicle)) # Sin filtro de tenant = bug de seguridad
```

Sistema de Alertas en Tiempo Real

1. Cada paquete TCP recibido dispara `asyncio.create_task(_check_alerts())`
2. Se cargan todas las reglas activas del tenant para ese vehículo
3. El valor raw se obtiene de `avl.io[io_id]` por nombre o ID numérico
4. Conversión: `eng = raw * scale_factor + offset`
5. Si condición cumplida y sin alerta abierta: crea AlertLog (respetando cooldown)

- 6. Si condición NO cumplida y hay alerta abierta: establece resolved_at
- 7. Publica en Redis pubsub para que el WebSocket lo retransmita al navegador

CMG
Telematics

MONITORIZACIÓN

Flota

Vehículos

Mapa

Alertas

Rutas

Analíticas

Mantenimiento

Geocercas

Eco-Driving

ADMINISTRACIÓN

Clientes

Usuarios

Administrador CMG
Superadmin

Cerrar sesión

Panel de Flota

Actualizado: 13:57:37

Actualizar

En línea
0 / 2

Con GPS
1

Ignición ON
1

Alertas activas
0

Mant. vencido
0

Ver analíticas

Flota (2)

Actividad

Camión Vacío Test 001

TEST-001

25 km/h

91 bar

24.3V

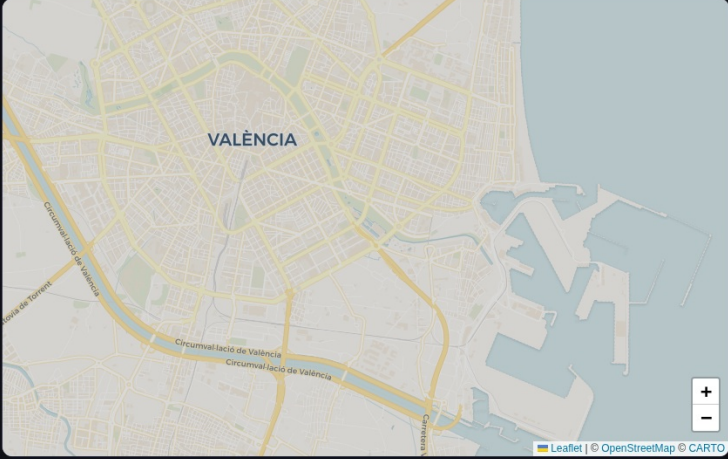
Ignición ON

OFFLINE

OT98976

Sin datos de posición

OFFLINE



Leaflet | OpenStreetMap © CARTO

Dashboard principal — KPIs, mapa de flota y eventos recientes

Frontend Next.js

Estructura del Proyecto

```

/opt/cmg-telematics/frontend/
├─ app/                                # Next.js App Router
│  ├─ page.tsx                         # / → redirect a /dashboard
│  ├─ layout.tsx                       # Root layout
│  ├─ globals.css                      # Variables CSS globales
│  ├─ login/page.tsx                  # Login email + password
│  ├─ dashboard/
│  │  ├─ layout.tsx                   # → AppShell
│  │  └─ page.tsx                     # KPIs, FleetMap, sidebar flota, eventos
│  ├─ vehicles/
│  │  ├─ layout.tsx
│  │  └─ page.tsx                     # Lista de vehículos con búsqueda
│  │     └─ [id]/page.tsx             # Detalle: telemetría, mapa, trips, DOUT, mantenimiento
│  ├─ map/
│  │  ├─ layout.tsx                   # AppShell overflow="hidden"
│  │  └─ page.tsx                     # Mapa full-screen tiempo real
│  ├─ alerts/
│  │  └─ page.tsx                     # Alertas activas + historial + acknowledge
│  ├─ trips/page.tsx                  # Rutas/viajes con TripMap
│  ├─ analytics/page.tsx              # Analíticas de flota
│  ├─ maintenance/page.tsx            # Tareas de mantenimiento + logs
│  ├─ geofences/page.tsx              # Geocercas con GeofenceDrawMap
│  ├─ ecodriving/page.tsx              # Scores de conducción eficiente
│  ├─ profile/page.tsx                # Perfil usuario + cambio contraseña
│  └─ admin/
│     ├─ layout.tsx
│     ├─ tenants/page.tsx             # CRUD tenants (solo superadmin)
│     ├─ users/page.tsx               # CRUD usuarios
│     ├─ vehicles/page.tsx            # Admin vehículos + asignación dispositivos
│     └─ variable-maps/page.tsx       # Mapeo variables IO
├─ components/
│  ├─ AppShell.tsx                    # Wrapper: Sidebar + AuthGuard + padding móvil
│  ├─ AuthGuard.tsx                   # Redirige a /login si no hay token
│  ├─ Sidebar.tsx                     # Navegación (desktop: sidebar / móvil: bottom tabs)
│  ├─ FleetMap.tsx                    # Mapa Leaflet, marcadores vehículos
│  ├─ TripMap.tsx                     # Mapa ruta coloreada por velocidad
│  ├─ VehiclePositionMap.tsx          # Mapa posición único vehículo
│  ├─ GeofenceDrawMap.tsx             # Mapa interactivo para geocercas
│  ├─ StatCard.tsx                    # Tarjeta KPI
│  ├─ Modal.tsx                       # Modal genérico
│  └─ Toast.tsx                       # Notificación temporal
├─ lib/
│  ├─ api.ts                          # Todas las funciones de API
│  ├─ websocket.ts                    # Hook useFleetWebSocket
│  └─ toast.ts                         # Hook useToast
├─ public/
│  ├─ manifest.json                   # PWA manifest
│  └─ icons/                          # Iconos PWA
├─ next.config.js                     # Config Next.js + PWA + rewrites
└─ package.json

```

Páginas Implementadas (15 rutas)

RUTA	DESCRIPCIÓN	ROL MÍNIMO
/dashboard	KPIs de flota, FleetMap, eventos recientes, WebSocket	viewer
/vehicles	Lista de vehículos con búsqueda y estado online/offline	viewer
/vehicles/[id]	Telemetría histórica (Recharts), mapa posición, trips, DOUT, mantenimiento	viewer
/map	Mapa full-screen tiempo real con filtros de flota	viewer
/alerts	Alertas activas + historial + acknowledge + configurar reglas	viewer
/trips	Listado de viajes con TripMap coloreada por velocidad	viewer
/analytics	Analíticas de flota: estadísticas, gráficas	viewer
/maintenance	Tareas de mantenimiento: vencidas, próximas, historial	viewer
/geofences	Geocercas: crear círculos/polígonos, eventos de entrada/salida	operator
/ecodriving	Scores de conducción eficiente por vehículo/conductor	viewer
/profile	Perfil de usuario + cambio de contraseña	any
/admin/tenants	CRUD de tenants con jerarquía (solo superadmin)	superadmin
/admin/users	CRUD de usuarios con roles	admin
/admin/vehicles	Admin vehículos: fabricante→cliente→vehículo + asignación IMEI	admin
/admin/variable-maps	Mapeo IO → unidades de negocio (pestañas: Plantillas / Excepciones)	admin

CMG
Telematics

MONITORIZACIÓN

Flota

Vehículos

Mapa

Alertas

Rutas

Analíticas

Mantenimiento

Geocercas

Eco-Driving

ADMINISTRACIÓN

Cientes

Usuarios

Administrador CMG
Superadmin

Cerrar sesión

Vehículos

0 en línea · 1 en movimiento · 2 total · 13:57:40

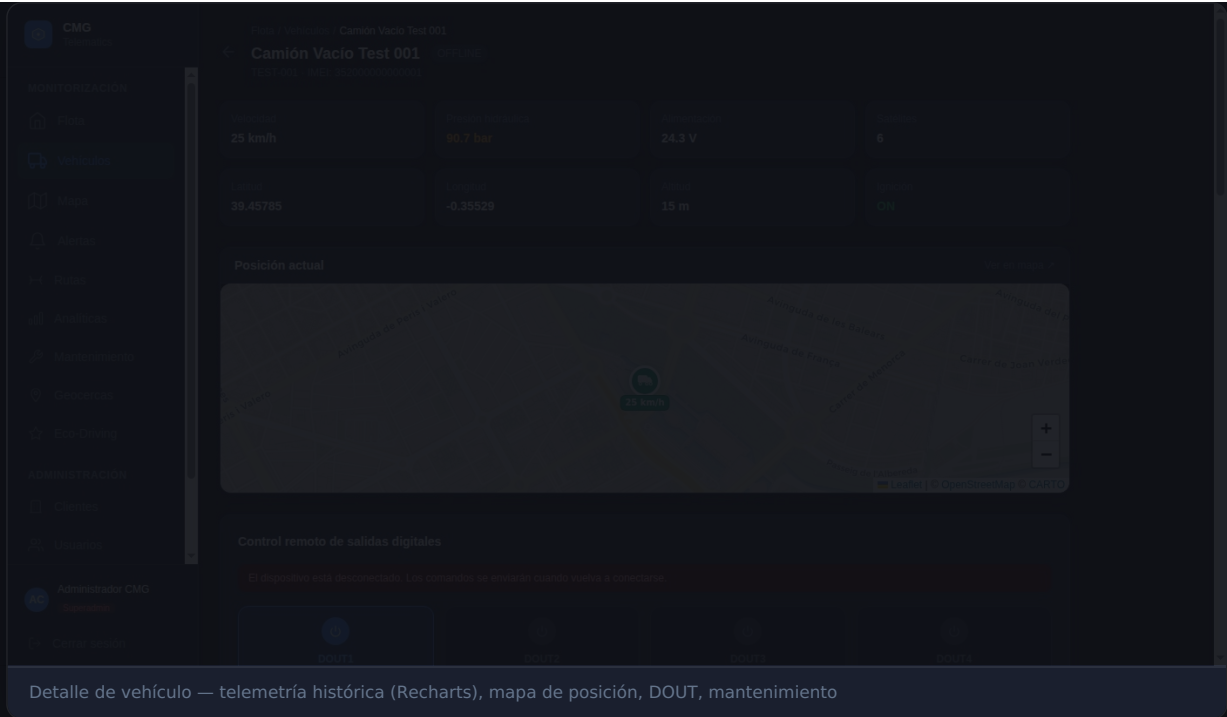
Actualizar

Buscar vehículo...

TodosEn líneaOffline

Vehículo	IMEI	Estado	Velocidad	Presión	Encendido	Última señal	
Camión Vacío Test 001 TEST-001	352000000000001	Offline	25 km/h	91 bar	ON	hace 17h	DetalleRutas
OT98976	-	Sin dispositivo	-	-	-	-	DetalleRutas

Lista de vehículos — búsqueda, estado online/offline, última señal



Componentes Principales

AppShell

Componente wrapper que incluye AuthGuard, Sidebar y gestiona el padding móvil. Todos los layouts de página usan `<AppShell>`. El mapa usa `<AppShell overflow="hidden">` para ocupar todo el viewport sin scroll.

Sidebar (Navegación adaptativa)

DESKTOP ($\geq 768\text{PX}$)

Sidebar izquierdo fijo de 220px con todos los items de navegación y logo CMG Telematics.

MÓVIL ($< 768\text{PX}$)

Bottom tab bar fijo de 64px: Flota, Vehículos, Mapa, Alertas (con badge rojo), Más. El tab "Más" abre un bottom sheet con accesos secundarios y admin.

FleetMap

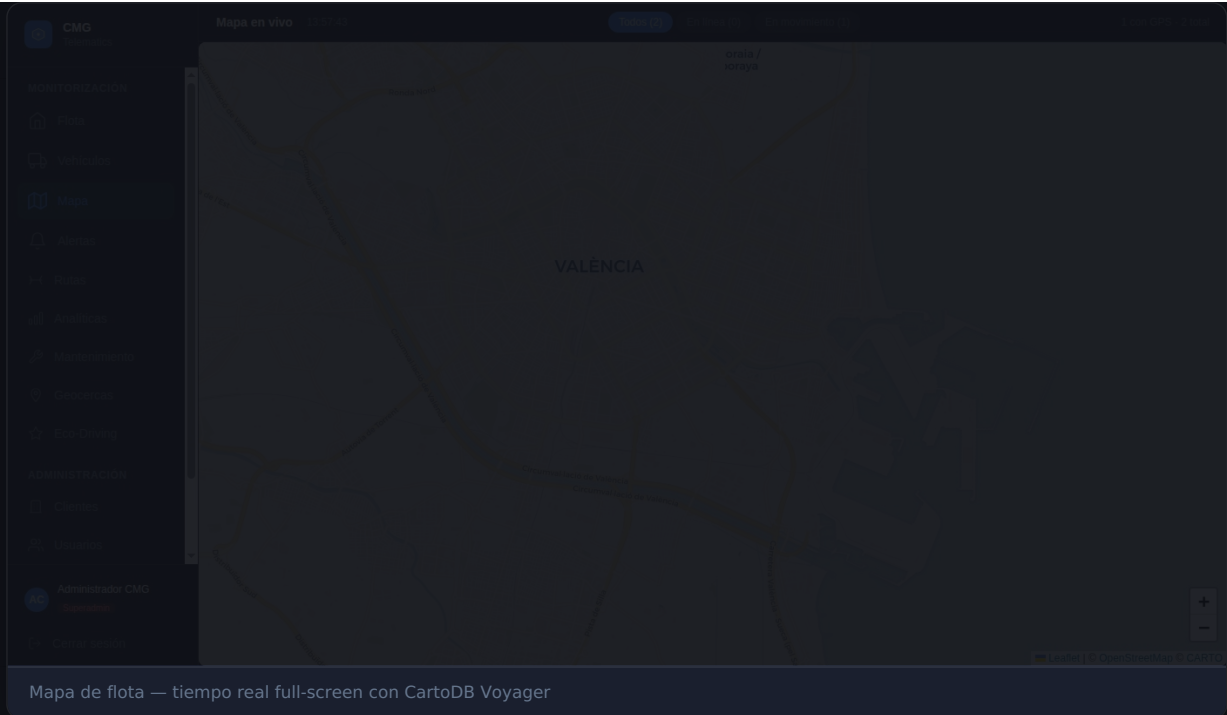
Mapa Leaflet con tiles CartoDB Voyager. Marcadores SVG de camión coloreados por estado (verde=online, gris=offline). Popup con nombre, estado, velocidad, última señal. Recibe actualizaciones en tiempo real vía WebSocket.

```
const TILE_URL = "https://{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}{r}.png";
// subdomains: 'abcd', maxZoom: 20, attribution: '© OpenStreetMap © CARTO'
```

Comandos DOUT — Seguridad crítica

Confirmación obligatoria antes de enviar DOUT

Los toggles de salida digital SIEMPRE muestran un modal de confirmación antes de ejecutar. Activar/desactivar una bomba hidráulica en campo sin confirmación puede causar accidentes. Esta validación no debe eliminarse.



Proxy API y WebSocket

El frontend hace fetch a `/api/...` que Next.js redirige al backend:

```
// next.config.js
async rewrites() {
  return [
    { source: '/api:path*', destination: 'http://localhost:8010/api:path*' },
    { source: '/health', destination: 'http://localhost:8010/health' },
  ];
}
```

El WebSocket no pasa por el proxy — conecta directamente con la IP pública:

```
// lib/websocket.ts
const WS_URL = `ws://213.210.20.183/ws/fleet?token=${token}`;
// La conexión va: navegador → Caddy :80 → FastAPI :8010 → /ws/fleet
```

Configuración PWA

La app es instalable como PWA (Progressive Web App). Configurada con `next-pwa 5.6.0` :

```
// next.config.js
const withPWA = require("next-pwa")({
  dest: "public",
  register: true,
  skipWaiting: true,
  disable: process.env.NODE_ENV === "development",
  runtimeCaching: [
    {
      urlPattern: /^https?\.*/api/v1/dashboard/fleet/,
      handler: "NetworkFirst", // Datos frescos con fallback a caché
      options: { cacheName: "fleet-cache", expiration: { maxAgeSeconds: 60 } }
    },
    {
      urlPattern: /^https?\.*/api/v1/vehicles/.*/last/,
      handler: "StaleWhileRevalidate", // Mostrar caché mientras actualiza
      options: { cacheName: "telemetry-last-cache", expiration: { maxAgeSeconds: 300 } }
    }
  ]
})
```

```
});
```

Service Worker y caché

El service worker cachea JS/CSS entre deploys. Si los usuarios ven versión antigua: Ctrl+Shift+R (desktop) o DevTools → Application → Service Workers → Unregister → recargar.

Ciclo de Build y Deploy — OBLIGATORIO

Cualquier cambio en el frontend **REQUIERE** este ciclo completo

El servicio corre en modo producción (next start). Los cambios NO son visibles hasta hacer el build y reiniciar el servicio.

```
# 1. Editar los ficheros fuente
# 2. Build de producción
cd /opt/cm-g-telematics/frontend
npm run build
# → Genera .next/ con el bundle optimizado

# 3. Reiniciar servicio
systemctl restart cm-g-telematics-frontend

# 4. Verificar
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/dashboard
# → 200
journalctl -u cm-g-telematics-frontend --since "1 minute ago"
```

Protocolo Teltonika FMC650

Protocolo Codec 8

El FMC650 usa **Teltonika Codec 8** sobre TCP persistente. El protocolo define el formato binario de los paquetes de telemetría AVL (Automatic Vehicle Location).

Handshake Inicial

```
# Cliente (FMC650) → Servidor:
Bytes 0-1: longitud del IMEI (uint16 big-endian) = 0x000F (15)
Bytes 2-16: IMEI en ASCII (15 caracteres numéricos)

# Servidor → Cliente:
0x01 = IMEI aceptado (dispositivo registrado en DB)
0x00 = IMEI rechazado (dispositivo desconocido o inactivo)
```

Estructura del Paquete Codec 8

OFFSET	TAMAÑO	TIPO	DESCRIPCIÓN
0	4 bytes	uint32 BE	Preamble: siempre 0x00000000
4	4 bytes	uint32 BE	Data Length (desde codec_id hasta último num_records)
8	1 byte	uint8	Codec ID: 0x08 (Codec 8)
9	1 byte	uint8	Number of Data 1 (N records)
10	variable	—	N × AVL Records
var	1 byte	uint8	Number of Data 2 (mismo que N)
var	4 bytes	uint32 BE	CRC-16/IBM (sobre bytes codec_id..num_data_2)

Estructura del AVL Record

OFFSET	TAMAÑO	TIPO	DESCRIPCIÓN
0	8	uint64 BE	Timestamp Unix en milisegundos
8	1	uint8	Priority: 0=low, 1=high, 2=panic
9	4	int32 BE	Longitud × 10,000,000 (signed, dividir para grados)
13	4	int32 BE	Latitud × 10,000,000 (signed)
17	2	int16 BE	Altitud (metros sobre nivel del mar)
19	2	uint16 BE	Ángulo (grados, 0=Norte, sentido horario)
21	1	uint8	Satélites GPS
22	2	uint16 BE	Velocidad (km/h)
24	1	uint8	Event IO ID (0=periódico, N=generado por IO N)
25	1	uint8	Total IO Count
26+	var	—	N1 IOs de 1 byte, N2 de 2 bytes, N4 de 4 bytes, N8 de 8 bytes

CRC-16/IBM

```
def crc16_ibm(data: bytes) -> int:
    # CRC calculado sobre bytes desde codec_id hasta num_data_2 (inclusive)
    crc = 0
    for byte in data:
        crc ^= byte
        for _ in range(8):
            if crc & 1:
                crc = (crc >> 1) ^ 0xA001
            else:
                crc >>= 1
    return crc
```

Servidor TCP asyncio

El servidor `TeltonikaServer` gestiona conexiones TCP persistentes con asyncio. Arranca como tarea asyncio en el startup de FastAPI:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()
    tcp_task = asyncio.create_task(teltonika_server.start())
    yield
    tcp_task.cancel()
```

Flujo de conexión por dispositivo

1. **Handshake:** lee 2+15 bytes IMEI, valida contra DB, acepta (0x01) o rechaza (0x00)
2. **Estado online:** marca device.online=True, registra writer en `device_registry` (Redis)
3. **Bucle de recepción:** lee 4 bytes preamble, si 0x00000000 es telemetría, si no es eco de comando
4. **Procesamiento:** parsea Codec 8, guarda en PostgreSQL, publica en Redis pubsub
5. **ACK:** envía uint32 con número de registros guardados
6. **Comandos:** después del ACK vacía la cola de comandos pendientes
7. **Desconexión:** marca device.online=False, limpia registros en memoria y Redis

Orden ACK → Comando (crítico)

Los comandos DOUT se envían DESPUÉS del ACK, nunca antes. Esto garantiza que el byte stream no mezcle el uint32 del ACK con los bytes del comando, lo que corrompería la sesión TCP.

Tabla de IDs de IO

IO ID	TAMAÑO	NOMBRE	DESCRIPCIÓN
1	1 byte	din1_ignition	Entrada digital 1 / Ignition (0=OFF, 1=ON)
2	1 byte	din2	Entrada digital 2
3	1 byte	din3	Entrada digital 3
4	1 byte	din4	Entrada digital 4
9	2 bytes	ain1_mv	Entrada analógica 1 (mV raw, 0-30000)
10	2 bytes	ain2_mv	Entrada analógica 2 (mV raw)
11	2 bytes	ain3_mv	Entrada analógica 3 (mV raw)
16	4 bytes	total_odometer_m	Odómetro total (metros acumulados)
21	1 byte	gsm_signal	Nivel señal GSM (0-5)

ID	Size	Signal Name	Description
24	2 bytes	speed_kmh	Velocidad (km/h, duplicado del GPS)
66	4 bytes	ext_voltage_mv	Tensión de alimentación externa (mV)
67	4 bytes	battery_mv	Batería interna del dispositivo (mV)
179	1 byte	dout1_status	Estado salida digital 1 (0=OFF, 1=ON)
180	1 byte	dout2_status	Estado salida digital 2
181	1 byte	dout3_status	Estado salida digital 3
182	1 byte	dout4_status	Estado salida digital 4
300-399	variable	CAN J1939	Señales CAN configurables del IFM CR2530 → almacenadas en io_data JSONB

Comandos Remotos DOUT

Para activar/desactivar una salida digital del FMC650 se usa el comando ASCII `setdigout` :

```
# Formato del comando enviado al dispositivo:
[2 bytes BE: longitud][comando ASCII]

# Comando: "setdigout X Y T"
#   X = máscara de bits (2^(output-1)): DOUT1=1, DOUT2=2, DOUT3=4, DOUT4=8
#   Y = valor: 1=ON, 0=OFF
#   T = duración en segundos (0=permanente)

# Ejemplos:
setdigout 1 1 0    # DOUT1 ON permanente
setdigout 2 0 0    # DOUT2 OFF permanente
setdigout 1 1 30   # DOUT1 ON durante 30 segundos

# Uso via API:
POST /api/v1/commands/send
{
  "imei": "352000000000001",
  "output": "DOUT1",
  "value": true,
  "duration_seconds": 0
}
```

Variable Maps — Arquitectura Two-Scope

El sistema de variable maps permite configurar cómo se interpretan los IOs raw del FMC650 en unidades de negocio (bar, rpm, °C...):

PLANTILLA DE FABRICANTE

`tenant_id set + vehicle_id null`

Aplica a todos los vehículos fabricados por ese fabricante. Define la conversión estándar para el modelo de máquina.

EXCEPCIÓN DE VEHÍCULO

`vehicle_id set + tenant_id null`

Anula la plantilla del fabricante para un vehículo específico. Útil cuando un vehículo tiene sensores distintos al estándar del modelo.

```
# Ejemplo: presión hidráulica
{
  "io_key": "ain1_mv",          # IO ID 9: entrada analógica 1
  "display_name": "Presión hidráulica",
  "unit": "bar",
  "scale_factor": 0.006,      # raw_mv * 0.006 = bar (0-180 bar en 0-30000 mV)
  "offset": 0.0,
  "alert_high": 300.0,        # Alerta si supera 300 bar
  "alert_low": 5.0,           # Alerta si baja de 5 bar (pérdida de presión)
  "data_type": "gauge"
}

# Resolver el mapa para un vehículo (plantilla + excepciones merged):
GET /api/v1/variable-maps/resolved?vehicle_id=<uuid>
```

CMG

MONITORIZACIÓN

Flota

Centros de datos

Mapas

Alertas

Notas

Análisis

Mantenimiento

Base de datos

Eventos

ADMINISTRACIÓN

Usuarios

Administrador CMG

Control de acceso

Variables IO

Plantillas de fabricante

Excepciones por vehículo

Las plantillas se aplican a todos los vehículos de una fabricante determinada. Configura aquí las variables IO asociadas de los modelos de máquina para una fabricante concreta.

Plantillas

MAX Equipment SL

Nueva plantilla

MAX Equipment SL

0 variables configuradas

No hay plantillas para esta fabricante. Añade la primera variable IO.

Admin Variable Maps — dos pestañas: Plantillas de fabricante / Excepciones por vehículo

Configuración Hardware FMC650

Configuración del Dispositivo FMC650

El **Teltonika FMC650** es un rastreador GPS/telemático para vehículos industriales. Se configura con el software **Teltonika Configurator** (Windows) vía cable USB o Bluetooth.

Parámetros de conexión al servidor

Server IP	213.210.20.183
Server Port	5027
Protocol	TCP
Codec	Codec 8
Record sending period	10-60 segundos (ajustar según necesidad)
Min period on stop	300 segundos
APN	Según operador SIM instalada

Configuración de IO

IO	FUNCIÓN	CONFIGURACIÓN
DIN1	Ignición	Conectar al positivo de contacto del vehículo
DIN2-4	Entradas auxiliares	Sensores digitales (presostatos, finales de carrera)
AIN1	Presión hidráulica	Sensor 0-10V o 4-20mA (via shunt) → 0-30000 mV
AIN2-3	Entradas analógicas	Temperatura, nivel, presión secundaria
DOUT1	Válvula/relay 1	Relay activo: ON = circuito hidráulico activo
DOUT2-4	Salidas auxiliares	Otros actuadores (bomba, válvula, señal)
CAN H/L	Bus CAN J1939	Conectar al conector OBD2/J1939 del vehículo

Registro en la plataforma

```
# Antes de conectar el dispositivo, registrar el IMEI en la plataforma:
POST /api/v1/admin/devices
Authorization: Bearer <admin_token>
{
  "imei": "352000000000001", # IMEI del dispositivo (15 dígitos, en pegatina del FMC650)
  "vehicle_id": "uuid-vehiculo",
  "model": "FMC650"
}
```

IMEI y seguridad

El servidor TCP rechaza (0x00) cualquier IMEI no registrado en la base de datos. Esto previene que dispositivos no autorizados envíen telemetría.

Verificar conexión

```
# Verificar que el dispositivo conectó:
curl http://localhost:8010/health
# → "active_devices": 1 si hay un dispositivo conectado

# Ver último registro en DB:
PGPASSWORD=cmg_pilot_2024 psql -U cmg -d cmg_telematics -c "
SELECT time, lat, lng, speed, ignition, ext_voltage_mv
FROM telemetry_record
ORDER BY time DESC
LIMIT 5;"

# Ver logs del TCP server:
journalctl -u cmg-telematics -f | grep "IMEI"
```

Bus CAN e IFM CR2530

El **IFM CR2530** es un controlador CAN J1939 que convierte señales hidráulicas (presión, temperatura, caudal) en tramas CAN que el FMC650 puede leer.

Configuración del bus CAN en FMC650

CAN baudrate	250 kbps (J1939 estándar)
CAN mode	SAE J1939
PGN monitoring	Configurar los PGNs de los parámetros del CR2530
IO IDs asignados	300-399 (rango CAN configurable en Teltonika)

Mapeo de señales CAN → Variable Maps

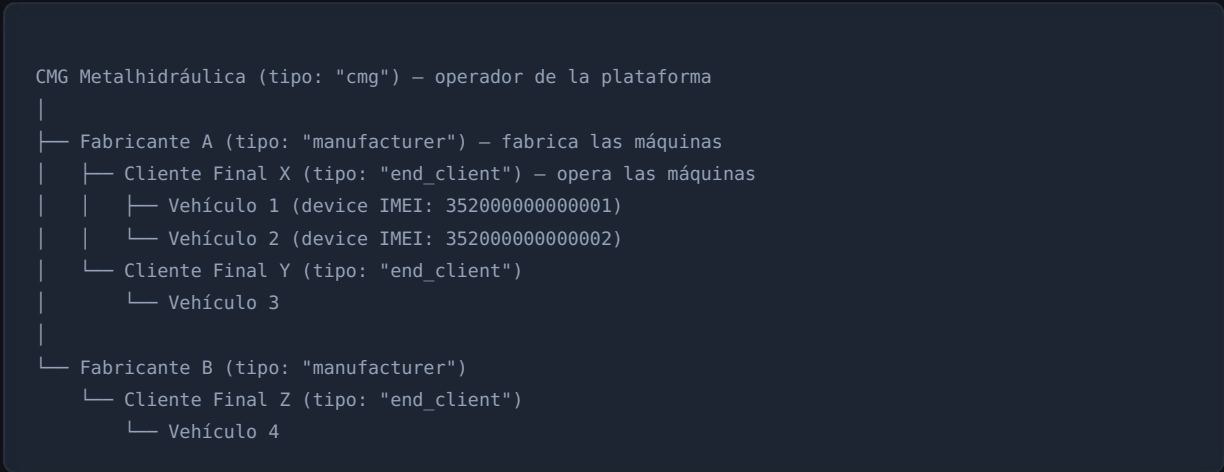
```
# Las señales CAN J1939 llegan como io_data["300"], io_data["301"], etc.
# Configurar Variable Map para interpretar:
{
  "io_key": "io_300",          # IO ID 300 del bus CAN
  "display_name": "Presión circuito principal",
  "unit": "bar",
  "scale_factor": 0.1,        # raw * 0.1 = bar (según datasheet CR2530)
  "offset": 0.0,
  "data_type": "gauge"
}

{
  "io_key": "io_301",          # IO ID 301
  "display_name": "Temperatura aceite hidráulico",
  "unit": "°C",
  "scale_factor": 0.25,        # raw * 0.25 - 40 = °C (según datasheet)
  "offset": -40.0,
  "data_type": "gauge"
}
```

CAPÍTULO 7

Administración de la Plataforma

Jerarquía de Tenants

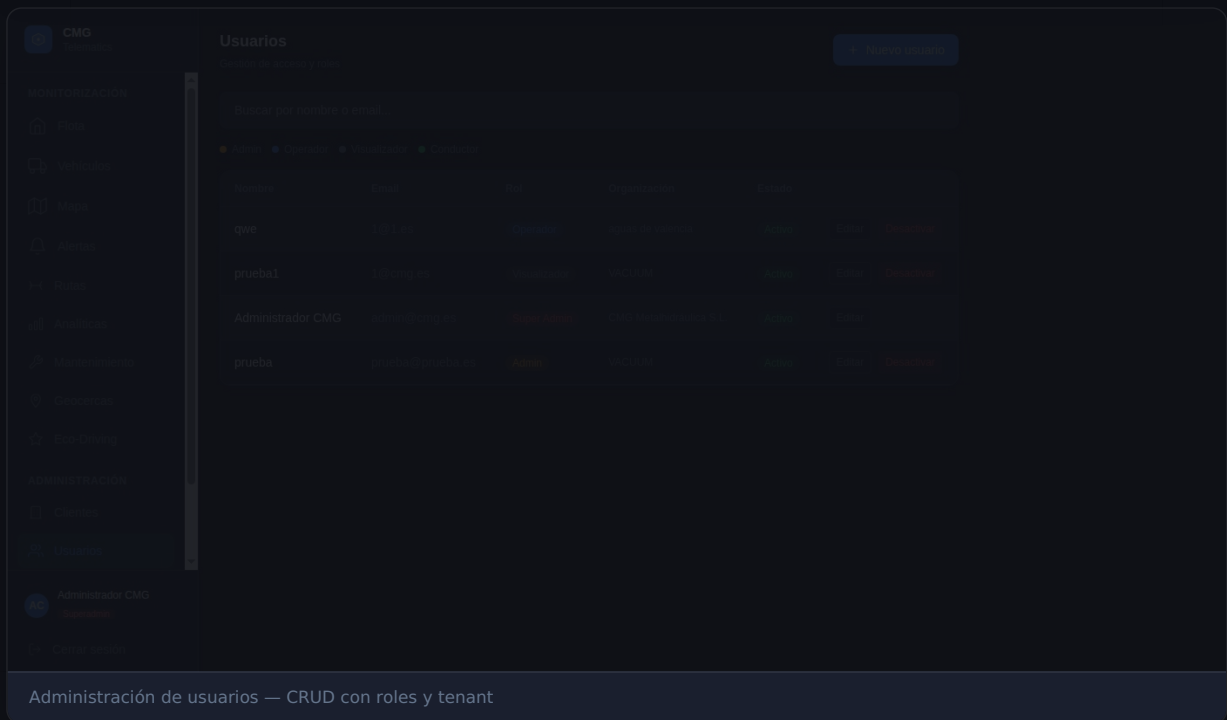
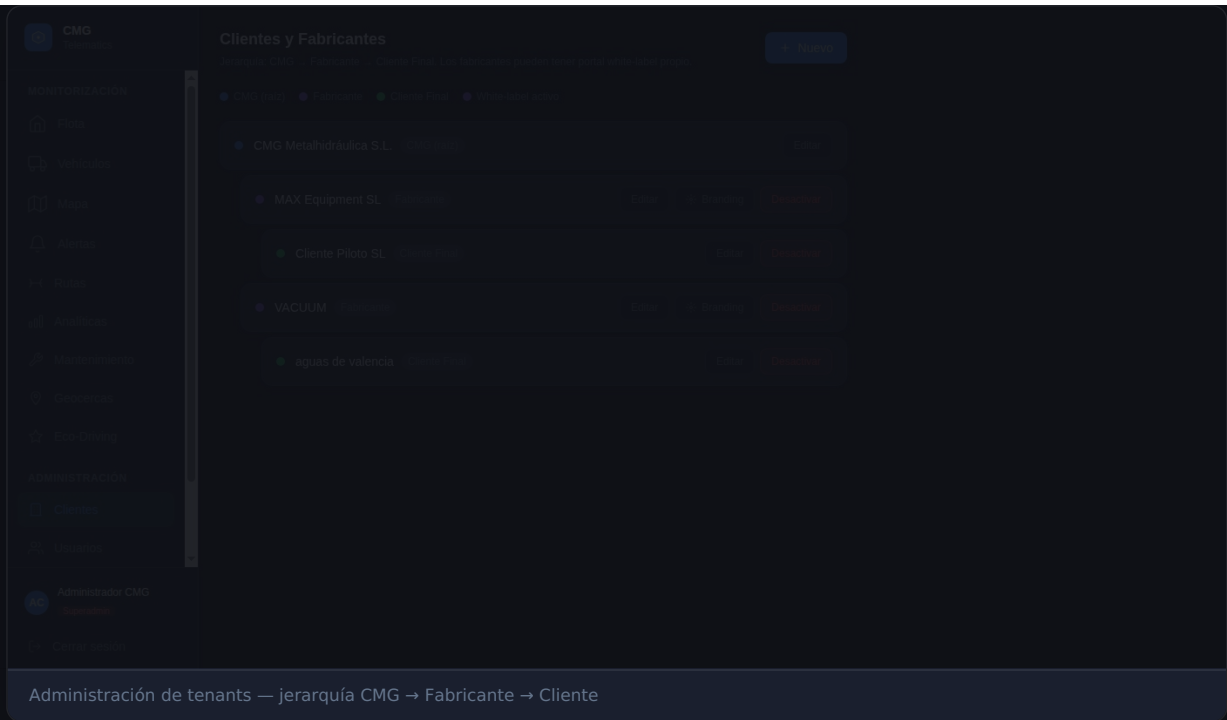


Reglas de la jerarquía:

- CMG (superadmin) ve todos los tenants, usuarios y vehículos
- Un Fabricante (admin) ve sus clientes y sus vehículos
- Un Cliente Final (operator/viewer) solo ve sus propios vehículos
- Un Conductor (driver) solo ve el vehículo asignado
- Los Variable Maps del fabricante se aplican a todos sus vehículos

Roles y Permisos

ROL	DESCRIPCIÓN	PERMISOS
SUPERADMIN	Operador CMG	Todo: gestión de tenants, usuarios, vehículos, configuración global
ADMIN	Fabricante	Gestionar sus clientes, vehículos, variable maps, reglas de alerta
OPERATOR	Cliente final (jefe de flota)	Ver vehículos, telemetría, enviar comandos DOUT, ver/ack alertas
VIEWER	Usuario de solo lectura	Ver vehículos, telemetría, alertas (sin enviar comandos)
driver	Conductor	Solo su vehículo asignado: posición y telemetría propia



Configuración Paso a Paso

1. Crear estructura de tenants

```
# Crear tenant fabricante (con superadmin token)
POST /api/v1/admin/tenants
{
  "name": "Fabricante Hidráulicos S.L.",
  "type": "manufacturer",
  "parent_id": "<id-tenant-cmg>",
  "brand_name": "FabriHidro",
  "brand_color": "#1D9E75"
}

# Crear tenant cliente final
POST /api/v1/admin/tenants
```

```
{
  "name": "Transportes García S.A.",
  "type": "end_client",
  "parent_id": "<id-tenant-fabricante>"
}
```

2. Crear usuarios

```
# Crear admin del fabricante
POST /api/v1/admin/users
{
  "email": "admin@fabricante.es",
  "password": "contraseña_segura",
  "full_name": "Admin Fabricante",
  "role": "admin",
  "tenant_id": "<id-tenant-fabricante>"
}

# Crear operador del cliente
POST /api/v1/admin/users
{
  "email": "jefe.flota@garcia.es",
  "password": "contraseña_segura",
  "full_name": "Jefe de Flota",
  "role": "operator",
  "tenant_id": "<id-tenant-cliente>"
}
```

3. Registrar vehículo y dispositivo

```
# Crear vehículo
POST /api/v1/admin/vehicles
{
  "name": "Camión Grúa 01",
  "license_plate": "1234 ABC",
  "vin": "VF1AA000000000001",
  "tenant_id": "<id-tenant-cliente>",
  "manufacturer_id": "<id-tenant-fabricante>"
}

# Registrar dispositivo FMC650
POST /api/v1/admin/devices
{
  "imei": "352000000000001",
  "vehicle_id": "<id-vehiculo>",
  "model": "FMC650"
}
```

4. Configurar Variable Maps (fabricante)

```
# Plantilla para todos los vehículos del fabricante
POST /api/v1/variable-maps
{
  "tenant_id": "<id-tenant-fabricante>",    # plantilla de fabricante
  "vehicle_id": null,
  "io_key": "ain1_mv",
  "display_name": "Presión hidráulica principal",
  "unit": "bar",
  "scale_factor": 0.006,
  "offset": 0.0,
  "alert_high": 300.0,
  "alert_low": 5.0,
  "data_type": "gauge"
}
```

5. Configurar Reglas de Alerta

```
POST /api/v1/alert-rules
{
  "tenant_id": "<id-tenant-cliente>",
  "vehicle_id": null,                # aplica a toda la flota
  "name": "Sobrepresión hidráulica",
  "io_key": "ain1_mv",
  "condition": "gt",
  "threshold": 300.0,
  "scale_factor": 0.006,
  "unit": "bar",
  "level": "high",
  "cooldown_minutes": 30,
  "display_name": "Presión hidráulica"
}

POST /api/v1/alert-rules
{
  "tenant_id": "<id-tenant-cliente>",
  "name": "Velocidad excesiva",
  "io_key": "speed",
  "condition": "gt",
  "threshold": 90.0,
  "unit": "km/h",
  "level": "medium",
  "cooldown_minutes": 5,
  "display_name": "Velocidad"
}
```

6. Configurar Tareas de Mantenimiento

```
POST /api/v1/maintenance/tasks
{
  "vehicle_id": "<id-vehiculo>",
  "name": "Cambio de aceite hidráulico",
  "description": "Cambiar aceite y filtros del circuito hidráulico",
  "trigger_type": "hours",
  "interval_value": 500,
  "next_due_hours": 500.0,
  "warn_before": 50.0,
  "pto_io_key": "dout1"             # contar horas cuando DOUT1 está activo (bomba en marcha)
}

POST /api/v1/maintenance/tasks
{
  "vehicle_id": "<id-vehiculo>",
  "name": "Revisión ITV",
  "trigger_type": "date",
  "next_due_date": "2026-12-15"
}
```


CAPÍTULO 8

Operación y Mantenimiento del Sistema

Comandos de Operación

Estado de servicios

```
# Ver estado de todos los servicios
systemctl status cmg-telematics           # Backend FastAPI + TCP server
systemctl status cmg-telematics-frontend # Frontend Next.js
systemctl status postgresql              # PostgreSQL 16
systemctl status redis-server            # Redis
systemctl status caddy                   # Reverse proxy
docker ps                                # Mosquitto MQTT

# Health check completo
curl http://localhost:8010/health
# → {"status":"ok","tcp_server":"running","db":"ok","redis":"ok","active_devices":0}
```

Reinicio de servicios

```
# Reiniciar backend (tras cambios Python)
systemctl restart cmg-telematics
journalctl -u cmg-telematics -f

# Reiniciar frontend (DESPUÉS del build)
cd /opt/cmg-telematics/frontend
npm run build                # Build obligatorio
systemctl restart cmg-telematics-frontend
journalctl -u cmg-telematics-frontend -f

# Reiniciar todos
systemctl restart cmg-telematics cmg-telematics-frontend caddy
```

Simulador FMC650 (desarrollo/pruebas)

```
# Lanzar simulador que imita un FMC650 real
cd /opt/cmg-telematics
python3 tests/simulate_fmc650.py

# El simulador:
# 1. Conecta a localhost:5027
# 2. Envía IMEI 352000000000001
# 3. Envía 5 registros AVL con GPS de Valencia, IO de prueba
# 4. Recibe ACK y confirma "setdigout" si se envía un comando DOUT

# Verificar datos en DB tras simulación:
PGPASSWORD=cmg_pilot_2024 psql -U cmg -d cmg_telematics -c "
SELECT time, lat, lng, speed, ignition, ext_voltage_mv, io_data
FROM telemetry_record
ORDER BY time DESC
LIMIT 5;"
```

Logs y Monitorización

```
# Logs en tiempo real
journalctl -u cmg-telematics -f                # Backend
```

```
journalctl -u cmg-telematics-frontend -f          # Frontend
journalctl -u cmg-telematics --since "1 hour ago" --no-pager

# Filtrar logs relevantes
journalctl -u cmg-telematics -f | grep -E "IMEI|ERROR|ALERT|WARN"

# Ver errores del último día
journalctl -u cmg-telematics --since "1 day ago" -p err

# Logs de Caddy
journalctl -u caddy -f

# Ver conexiones TCP activas al puerto 5027
ss -tnp | grep 5027
```

Acceso a Base de Datos

```
# Conectar a PostgreSQL
PGPASSWORD=cmg_pilot_2024 psql -U cmg -d cmg_telematics -h localhost

# Consultas útiles de monitorización:

-- Últimos 5 registros de telemetría
SELECT time, device_id, lat, lng, speed, ignition, ext_voltage_mv
FROM telemetry_record
ORDER BY time DESC
LIMIT 5;

-- Dispositivos conectados en las últimas 2 horas
SELECT d.imei, d.online, d.last_seen, v.name
FROM device d
LEFT JOIN vehicle v ON v.id = d.vehicle_id
WHERE d.last_seen > NOW() - INTERVAL '2 hours'
ORDER BY d.last_seen DESC;

-- Alertas activas (no resueltas)
SELECT al.fired_at, al.display_name, al.level, al.converted_value, al.unit, v.name
FROM alert_log al
JOIN vehicle v ON v.id = al.vehicle_id
WHERE al.resolved_at IS NULL
ORDER BY al.fired_at DESC;

-- Estadísticas de telemetría por hora con time_bucket
SELECT time_bucket('1 hour', time) AS hora, COUNT(*), AVG(speed)
FROM telemetry_record
WHERE time >= NOW() - INTERVAL '24 hours'
GROUP BY hora
ORDER BY hora DESC;

-- Estado de chunks TimescaleDB
SELECT chunk_name, range_start, range_end, is_compressed
FROM timescaledb_information.chunks
WHERE hypertable_name = 'telemetry_record'
ORDER BY range_start DESC
LIMIT 10;

-- Tamaño de la hypertable
SELECT hypertable_size('telemetry_record');
```

Resolución de Problemas

El dispositivo conecta pero no recibe ACK

```
# Verificar que el servidor TCP está escuchando
ss -tlnp | grep 5027
```

```
# Revisar logs del parser Codec 8
journalctl -u cmg-telematics -f | grep -E "Codec|CRC|parse"

# Verificar que el IMEI está registrado
PGPASSWORD=cmg_pilot_2024 psql -U cmg -d cmg_telematics -c "
SELECT imei, active, online FROM device WHERE imei = '352000000000001';"

# Si active=false: el dispositivo es rechazado (0x00) en el handshake
```

El frontend muestra versión antigua

```
# El service worker PWA puede cachear versiones antiguas
# Solución 1: forzar rebuild y reinicio
cd /opt/cmg-telematics/frontend
npm run build && systemctl restart cmg-telematics-frontend

# Solución 2 (usuario final): hard refresh
# Ctrl+Shift+R en desktop
# 0: DevTools → Application → Service Workers → Unregister → recargar
```

El WebSocket no recibe datos

```
# Verificar que Redis pubsub funciona
redis-cli -n 2 subscribe "telemetry:*"
# Debe recibir mensajes cuando llega telemetría

# Verificar que el WebSocket endpoint responde
curl -s -o /dev/null -w "%{http_code}" http://localhost:8010/health

# Verificar configuración Caddy para WebSocket
# El transporte debe ser HTTP/1.1 (no HTTP/2 – WebSocket lo requiere)
grep -A3 "ws/*" /etc/caddy/Caddyfile
```

Consultas lentas en telemetría

```
-- SIEMPRE filtrar por tiempo en telemetry_record
-- CORRECTO:
SELECT * FROM telemetry_record
WHERE device_id = '...'
  AND time >= NOW() - INTERVAL '24 hours' -- filtro temporal obligatorio
ORDER BY time DESC;

-- INCORRECTO (timeout garantizado):
SELECT * FROM telemetry_record WHERE device_id = '...'; -- sin filtro tiempo

-- Verificar que el índice se usa:
EXPLAIN ANALYZE
SELECT time, speed FROM telemetry_record
WHERE device_id = '...' AND time >= NOW() - INTERVAL '1 hour'
ORDER BY time DESC;
```

Tabla de errores de API

CÓDIGO HTTP	ERROR CODE	CAUSA	SOLUCIÓN
401	INVALID_CREDENTIALS	Email/password incorrectos	Verificar credenciales
401	Token inválido	JWT expirado o malformado	Hacer re-login para obtener nuevo token
403	PERMISSION_DENIED	Rol insuficiente	Usar cuenta con rol adecuado
403	TENANT_MISMATCH	Intento de acceder a datos de otro	Bug de seguridad — reportar

tenant			
404	VEHICLE_NOT_FOUND	Vehículo no existe o no pertenece al tenant	Verificar UUID del vehículo
409	DEVICE_OFFLINE	Comando enviado a dispositivo no conectado	Esperar a que el dispositivo se conecte
500	COMMAND_FAILED	Error interno al enviar comando	Ver logs del backend

CMG
Telematics

MONITORIZACIÓN

Flota

Vehículos

Mapa

Alertas

Rutas

Analíticas

Mantenimiento

Geocercas

Eco-Driving

ADMINISTRACIÓN

Clientes

Usuarios

Administrador CMG
Superadmin

Cerrar sesión

Historial de Rutas

Rutas detectadas por ciclos encendido/apagado

Camión Vacío Test 001

03/15/2026

03/22/2026

Buscar rutas

Sin rutas

No se detectaron rutas en el periodo seleccionado

Busca rutas para empezar

Módulo de viajes — listado con duración, distancia y ruta coloreada por velocidad

